Анализ данных с использованием языка Python

Пояснения к коду. Семинар 6

Pandas: типы, даты, пропуски, объединение таблиц

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-06-pandas-cleaning.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинарам 1–5, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/pandas.md`.

1 Две таблицы и три вопроса

Ячейка 1. Две таблицы

```
1 import numpy as np
2 import pandas as pd
3
4 dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id')  # ↩
  ↪ ДТП Брянской области
5 people = pd.read_csv('../datasets/legal/dtp-bryansk-participants.csv')  # ↩
  ↪ участники этих ДТП
6 dtp.shape, people.shape  # строки и столбцы таблиц
```

– `import numpy as np` — NumPy нужен ради `np.nan` в разделе 5.

– Строка 5 — таблица участников без `index_col`: своего номера у участника нет, pandas нумерует строки с нуля.

– `dtp.shape, people.shape` — размеры обеих таблиц.

Ячейка 2. Первые участники

```
1 people.head(3)  # первые три участника
```

– `people.head(3)` — первые три строки; `NaN` — пропуск.

2 Типы

Ячейка 3. Типы столбцов

```
1 people.dtypes  # типы столбцов участников
```

– `people.dtypes` — тип каждого столбца. Столбцы с пропусками и целыми по смыслу числами прочитаны как `float64`.

Ячейка 4. Целый тип и пропуски: намеренная ошибка

```
people['vehicle_year'].astype(int)  # целый тип и пропуски: ошибка
```

– `astype(int)` — обычный целый тип не хранит пропуски, поэтому `IntCastingNaNError`.

Ячейка 5. Целый тип с пропусками

```
people['vehicle_year'].astype('Int64').tail(4)  # годы выпуска целыми числами
```

– `astype('Int64')` — целый тип pandas, который хранит пропуски; имя с большой буквы и в кавычках. Пропуск печатается как `<NA>`.

3 Даты

Ячейка 6. dt у текста: намеренная ошибка

```
dtp['datetime'].dt.year  # dt у текста: ошибка
```

– `.dt` — доступ к частям даты; у текстового столбца его нет, поэтому `AttributeError`.

Ячейка 7. Текст в дату

```
dtp['datetime'] = pd.to_datetime(dtp['datetime'])  # текст становится датой
dtp['datetime'].head(3)
```

– `pd.to_datetime(dtp['datetime'])` — столбец текста в столбец дат; результат присвоен в тот же столбец. Тип `datetime64[us]` — дата и время с точностью до микросекунды.

Ячейка 8. День недели

```
dtp['datetime'].dt.day_of_week.value_counts().sort_index()  # ДТП по дням ↩
↪ недели
```

– `.dt.day_of_week` — номер дня недели: понедельник — 0, воскресенье — 6.

– `value_counts().sort_index()` — число ДТП по дням в порядке номеров.

Ячейка 9. Название дня

```
dtp['datetime'].dt.day_name().value_counts().head(3)  # три самых ↩
↪ аварийных дня
```

– `.dt.day_name()` — название дня по-английски; это метод, со скобками.

Ячейка 10. Час

```
dtp['datetime'].dt.hour.value_counts().head()  # пять самых аварийных часов
```

– `.dt.hour` — час от 0 до 23; `value_counts().head()` — пять самых частых часов.

Ячейка 11. Период

```
holidays = (dtp['datetime'] >= '2025-01-01') & (dtp['datetime'] < '2025-01-09')  # маска: праздничные дни января
holidays.sum()
```

– `dtp['datetime'] >= '2025-01-01'` — столбец дат сравнивается со строкой даты: pandas сам переводит строку в дату. Дата без времени — полночь.

– `< '2025-01-09'` — правая граница — следующий день со знаком «меньше»: так в период попадает весь последний день.

Ячейка 12. Разность дат

```
span = dtp['datetime'].max() - dtp['datetime'].min()  # от первого ДТП до последнего
span, span.days
```

– `max() - min()` — разность двух дат: промежуток `Timedelta`. `span.days` — целое число дней в нём.

Ячейка 13. Дата плюс промежуток

```
pd.Timestamp('2025-01-01') + pd.Timedelta(days=8)  # конец праздничных дней
```

– `pd.Timestamp('2025-01-01')` — одна дата; `pd.Timedelta(days=8)` — промежуток в восемь дней, единица названа явно. Сумма — новая дата.

Ячейка 14. Даты при загрузке

```
dated = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id',
                    parse_dates=['datetime'])  # даты при чтении
dated['datetime'].head(2)
```

– `parse_dates=['datetime']` — прочитать столбцы из списка сразу датами.

4 Пропуски

Ячейка 15. Пропуски по столбцам

```
people.isna().sum()  # пропуски по столбцам
```

– `people.isna()` — таблица той же формы из `True` и `False`: `True` там, где пропуск. `.sum()` — сумма по каждому столбцу.

Ячейка 16. Доля пропусков

```
people['experience'].isna().mean().round(3)  # доля пропусков стажа
```

– `isna().mean()` — доля пропусков в столбце.

Ячейка 17. У кого пропуск

```
people.loc[people['experience'].isna(), 'role'].value_counts().head(4)  # ↩
↪ у кого стаж не указан
```

– `people.loc[people['experience'].isna(), 'role']` — роли только тех строк, где стаж пропущен; `value_counts()` — сколько каждой роли.

Ячейка 18. Указано и всего

```
people['experience'].mean().round(1), people['experience'].count(), ↩
↪ len(people)  # среднее, указано, всего
```

– `mean()` пропуски пропускает; `count()` — число указанных значений; `len(people)` — число всех строк.

Ячейка 19. Заполнение

```
dtp['road_defect'] = dtp['road_defect'].fillna('не установлены')  # ↩
↪ пропуск получает название
dtp['road_defect'].value_counts().head(3)
```

– `fillna('не установлены')` — новый столбец, где пропуски заменены этим значением; он присвоен обратно в тот же столбец.

Ячейка 20. Удаление

```
len(people.dropna()), len(people.dropna(subset=['experience']))  # строк ↩
↪ после удаления
```

– `people.dropna()` — новая таблица без строк, где есть хотя бы один пропуск в любом столбце.

– `dropna(subset=['experience'])` — пропуски проверяются только в перечисленных столбцах.

– Результат никуда не присвоен, поэтому `people` не меняется.

Ячейка 21. Кто остался после dropna

```
people.dropna()['role'].value_counts()  # кто остался после dropna
```

– `people.dropna()['role'].value_counts()` — роли в таблице после удаления: цепочка без промежуточной переменной.

5 Коды вместо значений и дубликаты

Ячейка 22. Сводка стажа

```
people['experience'].describe().round(1)  # сводка стажа
```

– `describe().round(1)` — сводка одного столбца с одним знаком после точки.

Ячейка 23. Самые большие значения

```
people['experience'].value_counts().sort_index().tail(7)  # самые большие ↩
↪ значения стажа
```

– `value_counts().sort_index().tail(7)` — частоты в порядке значений стажа и семь последних, самых больших.

Ячейка 24. Коды в пропуск

```
people['experience_clean'] = people['experience'].replace([95, 96, 99], ↩
↪ np.nan)  # коды становятся пропусками
people['experience'].mean().round(1), ↩
↪ people['experience_clean'].mean().round(1)  # средний стаж до и после
```

– `replace([95, 96, 99], np.nan)` — заменить каждое значение из списка пропуском. Результат записан в новый столбец `experience_clean`, исходный `experience` не меняется.

– Строка 2 — средний стаж по исходному и по очищенному столбцу.

Ячейка 25. Год выпуска 1

```
people.loc[people['vehicle_year'] == 1, ['dtp_id', 'role', 'brand', ↩
↪ 'vehicle_year']]  # год выпуска 1
```

– `people.loc[маска, [...]]` — строки с годом выпуска 1 и четыре столбца.

Ячейка 26. Повторы

```
dtp.duplicated().sum(), people.duplicated().sum()  # повторы в двух таблицах
```

– `duplicated()` — маска: `True` у строки, которая целиком повторяет одну из предыдущих. `sum()` — число таких строк.

Ячейка 27. Как выглядят повторы

```
people[people.duplicated(keep=False)].head(6)  # как выглядят повторы
```

– `duplicated(keep=False)` — отмечает все строки каждой группы одинаковых, и первую тоже: так видны группы целиком.

Ячейка 28. Проверка по таблице ДТП

```
dtp.loc[133, 'participants_count'], (people['dtp_id'] == 133).sum()  # ↩
↪ участников по двум таблицам
```

– `dtp.loc[133, 'participants_count']` — число участников ДТП 133 по таблице ДТП.

– `(people['dtp_id'] == 133).sum()` — число строк этого ДТП в таблице участников.

Ячейка 29. Таблица машин

```
vehicles = people.dropna(subset=['vehicle_no'])[['dtp_id', 'vehicle_no', ↩
↪ 'brand', 'vehicle_year']]  # участники в машинах
vehicles = vehicles.drop_duplicates(ignore_index=True)  # одна строка на ↩
↪ машину
len(vehicles)
```

– `dropna(subset=['vehicle_no'])` — участники в машинах: у пешеходов номера машины нет. Затем четыре столбца о машине.

– `drop_duplicates(ignore_index=True)` — удалить повторы строк: у людей одной машины эти столбцы одинаковы, остаётся строка на машину. `ignore_index=True` — пронумеровать строки заново с нуля.

6 Объединение таблиц

Ячейка 30. Без ключа: намеренная ошибка

```
people.merge(dtp)  # ключ не назван: ошибка
```

– `people.merge(dtp)` — без ключа pandas ищет столбцы с одинаковыми именами в обеих таблицах; их нет, поэтому `MergeError`.

Ячейка 31. Объединение по ключу

```
both = people.merge(dtp, left_on='dtp_id', right_index=True)  # участник и его ДТП
both.shape
```

– `left_on='dtp_id'` — ключ левой таблицы, `people`: столбец с номером ДТП.

– `right_index=True` — ключ правой таблицы, `dtp`: её индекс.

– Каждой строке участника приставляется строка его ДТП. `both.shape` — проверка: строк столько же, сколько участников.

Ячейка 32. Столбцы двух таблиц

```
both[['dtp_id', 'role', 'health', 'datetime', 'district', 'light']].head(3)  # столбцы из двух таблиц
```

– Список столбцов из обеих исходных таблиц: в объединённой они стоят рядом.

Ячейка 33. inner и left

```
fatal = dtp[dtp['dead_count'] > 0]  # ДТП с погибшими
inner = people.merge(fatal, left_on='dtp_id', right_index=True)  # только участники этих ДТП
left = people.merge(fatal, left_on='dtp_id', right_index=True, how='left')  # все участники
len(inner), len(left), left['district'].isna().sum()
```

– `fatal` — только ДТП с погибшими.

– Строка 2 — объединение по умолчанию, оно же «inner»: остаются участники, у которых нашлось ДТП в таблице `fatal`.

– Строка 3 — `how='left'`: остаются все строки левой таблицы; у участников без пары столбцы `fatal` пустые.

– `left['district'].isna().sum()` — сколько строк осталось без пары.

Ячейка 34. Таблицы друг под другом

```
msk = pd.read_csv('../datasets/legal/dtp-moscow-region.csv', index_col='id',
                  parse_dates=['datetime'])  # ДТП Московской области
bryansk = dtp.copy()  # копия для склейки
bryansk['region'] = 'Брянская'
msk['region'] = 'Московская'
two = pd.concat([bryansk, msk])  # таблицы друг под другом
recent = two[two['datetime'].dt.year >= 2022]  # общие годы
(recent['dead_count'] > 0).groupby(recent['region']).agg(['size', 'mean']).round(3)
```

– Строки 1–2 — таблица Московской области, даты сразу датами.

– `bryansk = dtp.copy()` — копия, чтобы новый столбец не попал в `dtp`. `bryansk['region'] = 'Брянская'` — столбец с одним значением во всех строках.

– `pd.concat([bryansk, msk])` — сложить таблицы из списка друг под другом; столбцы совпадают по именам.

– Последняя строка — маска ДТП с погибшими группируется по региону: число ДТП и доля, как на семинаре 5.

7 Мини-анализ и итоги

Ячейка 35. Ночные часы

```
night_hours = dtp['datetime'].dt.hour.between(0, 5)  # маска: ночные часы
with_dead = dtp['dead_count'] > 0  # маска: ДТП с погибшими
night_hours.mean().round(3), with_dead[night_hours].mean().round(3), with_dead[~night_hours].mean().round(3)
```

– `dt.hour.between(0, 5)` — маска ДТП с полуночи до шести утра, обе границы включены.

– `with_dead[night_hours]` — маска ДТП с погибшими только среди ночных; её среднее — доля. `~night_hours` — остальные часы.

Ячейка 36. Стаж водителей

```
drivers = both[both['role'] == 'Водитель']  # водители с их ДТП
drivers.groupby('severity')['experience_clean'].agg(['size', 'median'])  # водители и медиана стажа
```

– `both[both['role'] == 'Водитель']` — водители из объединённой таблицы: у каждого есть тяжесть его ДТП.

– `groupby('severity')['experience_clean'].agg(['size', 'median'])` — по тяжести число водителей и медиана очищенного стажа.

Ячейка 37. Пешеходы в тёмное время

```
pedestrians = both[both['role'] == 'Пешеход']  # пешеходы с их ДТП
dark = pedestrians['light'].str.contains('темное')  # маска: тёмное время
(pedestrians['health'] == 'Погиб').groupby(dark).agg(['size', 'mean']).round(3)  # пешеходы и доля погибших
```

– `pedestrians['light'].str.contains('темное')` — маска тёмного времени для каждого сбитого пешехода.

– `(pedestrians['health'] == 'Погиб').groupby(dark)` — маска погибших группируется по маске `dark`: две группы, `False` и `True`. `agg(['size', 'mean'])` — число пешеходов и доля погибших.

Ячейка 38. В файл без индекса

```
both.to_csv('dtp-bryansk-joined.csv', index=False)  # объединённая таблица в файл
```

– `to_csv(..., index=False)` — записать таблицу без индекса: номер строки в файле не нужен.